شتاب‌دهنده‌های سخت‌افزاری

دانشکده مهندسی و علوم کامپیوتر

بهار ۱۴۰۳-۱۴۰۴

استاد درس: دارا رحمتی

طراحی تمرین: زهرا تاکی و متین فیروزبخت

تمرین پنجم

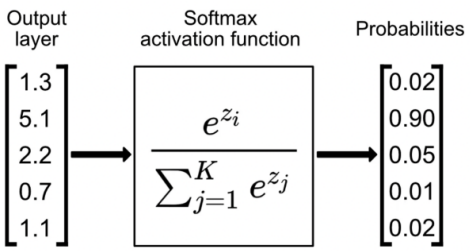
تاریخ انتشار : ۱۴۰۴/۰۲/۲۴ - تاریخ تحویل : ۱۴۰۴/۰۳/۰۲

معرفی

کودا[۱] یک پلتفرم محاسباتی و مدل برنامه‌نویسی است که امروزه به عنوان یکی از ابزارهای کلیدی در دنیای محاسبات با عملکرد بالا شناخته می‌شود. کودا توسط شرکت انویدیا[۲] توسعه یافته و به برنامه‌نویسان این امکان را می‌دهد تا از قدرت پردازش موازی کارت‌های گرافیک[۳] برای اجرای سریع‌تر برنامه‌ها بهره ببرند. با استفاده از کودا، می‌توان بخش‌های سنگین محاسباتی را به جای واحد پردازش مرکزی[۴] روی GPU اجرا کرد که باعث افزایش چشمگیر سرعت پردازش در کاربردهایی مانند یادگیری ماشین، پردازش تصویر و شبیه‌سازی‌های علمی می‌شود. این پلتفرم از زبان برنامه‌نویسی C/C++ پشتیبانی می‌کند و توسعه‌دهندگان می‌توانند با نوشتن کرنل‌ها[۵] عملیات موازی را کنترل کنند. ساختار سلسله‌مراتبی اجرای برنامه‌ها در کودا شامل بلاک‌ها[۶] و نخ‌ها[۷] است که امکان بهره‌گیری مؤثر از منابع سخت‌افزاری را فراهم می‌کند.

میتوانید برای آشنایی و یادگیری کودا از منابع قرار گرفته در قسمت منابع تمرین استفاده کنید.

تابع فعال‌سازی سافت‌مکس[۸]



تصویر ۱- تابع سافت‌مکس در شبکه‌های عصبی

۱. CUDA
۲. NVIDIA
۳. GPU
۴. CPU
۵. kernels
۶. Blocks
۷. Thread
۸. Softmax Activation Function

تابع سافت‌مکس که اغلب در لایه نهایی شبکه‌های عصبی برای انجام وظایف طبقه‌بندی استفاده می‌شود، مقادیر خام خروجی (که به آن‌ها logits نیز گفته می‌شود) را به احتمال‌هایی تبدیل می‌کند که قابل تفسیر هستند. این تبدیل با گرفتن نمایی[۹] از هر خروجی و سپس نرمال‌سازی آن‌ها با تقسیم بر مجموع کل نمایی‌ها انجام می‌شود. این فرآیند نشان داده شده در تصویر ۱، تضمین می‌کند که خروجی‌ها در بازه (۰ تا ۱) قرار بگیرند و مجموع آن‌ها برابر ۱ شود؛ به این ترتیب، خروجی‌ها به عنوان احتمال قابل تفسیر خواهند بود.

فرمول ریاضی تابع سافت‌مکس به صورت زیر است:

$$Softmax(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

در این فرمول، z_i ورودی مربوط به کلاس i و مخرج، مجموع نمایی‌های تمام نمرات خام مربوط به کلاس‌های مختلف است. برای درک بهتر، تصور کنید یک شبکه عصبی وظیفه تشخیص ارقام دست‌نویس از ۰ تا ۹ را دارد. لایه نهایی این شبکه ممکن است برداری شامل ۱۰ عدد خروجی دهد که هر عدد به یک رقم مربوط است. اما این اعداد احتمال نیستند. در اینجا تابع سافت‌مکس وارد عمل می‌شود و این بردار را به یک توزیع احتمالاتی تبدیل می‌کند که مشخص می‌کند هر عدد با چه احتمالی درست است.

از نظر کاربرد، تابع سافت‌مکس عمدتاً در مسائل طبقه‌بندی چندکلاسه[۱۰] استفاده می‌شود. از تشخیص تصویر و پردازش زبان طبیعی[۱۱] گرفته تا سامانه‌های توصیه‌گر، توانایی آن در مدیریت کلاس‌های مختلف به‌صورت همزمان، آن را به ابزاری ضروری تبدیل کرده است. برای مثال، در یک مدل شبکه عصبی که وظیفه تشخیص نوع میوه را دارد، سافت‌مکس احتمال اینکه تصویر ورودی مربوط به یک سیب، پرتقال یا موز باشد را مشخص می‌کند، به‌طوری که مجموع این احتمال‌ها برابر با ۱ خواهد بود.

عملکرد، ویژگی‌ها و مزایای تابع سافت‌مکس

عملکرد این تابع به شرح زیر است:

۱. **ورودی:** یک بردار از اعداد حقیقی را به عنوان ورودی دریافت می‌کند که از لایه نهایی شبکه عصبی آمده‌اند.

۲. **نمایی کردن:** هر عنصر در این بردار با استفاده از عدد e (تقریباً برابر ۲.۷۱۸) به توان رسانده می‌شود تا تمام مقادیر مثبت شوند.

---

۹. Exponential
۱۰. Multi-class classification
۱۱. NLP

۳. **نرمال‌سازی:** سپس این مقادیر نمایی‌شده بر مجموع کل آن‌ها تقسیم می‌شوند تا خروجی نهایی به‌دست آید که مجموع آن ۱ است.

در تصویر ۲ و ۳ میتوانید پیاده‌سازی پایتون و خروجی را مشاهده کنید.

Python

```python
from math import exp

def softmax(input_vector):
    # Calculate the exponent of each element in the input vector
    exponents = [exp(i) for i in input_vector]

    # Correct: divide the exponent of each value by the sum of the exponents
    # and round off to 3 decimal places
    sum_of_exponents = sum(exponents)
    probabilities = [round(exp(i) / sum_of_exponents, 3) for i in exponents]

    return probabilities

print(softmax([3.2, 1.3, 0.2, 0.8]))
```

تصویر ۲- پیاده‌سازی سافت‌مکس پایتون

```
Last executed at 2024-03-05 15:07:57 in 3ms
[1425560482.532, 1.239, 0.107, 0.293]
```

تصویر ۳- خروجی

ویژگی‌های تابع سافت‌مکس:

- **محدوده خروجی:** خروجی‌های تابع سافت‌مکس همیشه بین ۰ و ۱ هستند، که ویژگی ضروری برای احتمال بودن است.
- **جمع خروجی‌ها:** مجموع تمام خروجی‌ها برابر ۱ است.
- **قابلیت تفسیر:** سافت‌مکس خروجی‌های خام مدل را به احتمال‌هایی قابل فهم تبدیل می‌کند، که تحلیل و بررسی پیش‌بینی‌ها را ساده‌تر می‌سازد.

تابع سافت‌مکس مزایای متعددی دارد که برخی از آن‌ها عبارت‌اند از:

- **توزیع احتمالاتی**[۱۲]**:** سافت‌مکس یک توزیع احتمال دقیق برای هر کلاس تولید می‌کند که ارزیابی میزان اطمینان شبکه را ممکن می‌سازد.

---

۱۲ . Probability distribution

- **قابلیت تفسیر**[۱۳]: احتمال‌ها قابل فهم‌تر و قابل گزارش‌تر از خروجی‌های خام هستند و تحلیل شبکه را ساده‌تر می‌کنند.
- **پایداری عددی**[۱۴]: پیاده‌سازی سافت‌مکس (با تکنیک‌هایی مانند کم‌کردن بیشینه مقدار) پایداری عددی مناسبی دارد و آن را برای آموزش کارآمد شبکه‌های عصبی مناسب می‌سازد.

مقایسه با سایر توابع فعال‌سازی

بر خلاف توابعی مانند سیگموئید[۱۵] یا واحد خطی تصحیح‌شده[۱۶] ، که معمولاً در لایه‌های پنهان برای طبقه‌بندی دودویی یا اعمال تبدیلات غیرخطی استفاده می‌شوند، سافت‌مکس منحصراً برای لایه خروجی در مسائل طبقه‌بندی چندکلاسه مناسب است. سیگموئید خروجی‌ها را بین ۰ و ۱ فشرده می‌کند اما تضمینی برای جمع شدن خروجی‌ها تا ۱ وجود ندارد. همچنین ReLU به دلیل حل مشکل ناپدید شدن گرادیان معروف است. این ویژگی‌ها نقش سافت‌مکس در سناریوهای طبقه‌بندی را برجسته می‌کند.

چرا از کودا برای پیاده‌سازی سافت‌مکس استفاده می‌کنیم؟

همانطور که قبلا گفتیم تابع سافت‌مکس به‌طور گسترده در مسائل طبقه‌بندی چندکلاسه در شبکه‌های عصبی به‌کار می‌رود و شامل دو عملیات محاسباتی سنگین است:

۱. محاسبه نمایی عناصر ورودی
۲. نرمال‌سازی آن‌ها با مجموع کل

نکته مهم اینجاست که این تابع ذاتاً ماهیتی ترتیبی[۱۷] دارد؛ چرا که برای محاسبه مقدار نهایی هر عنصر، ابتدا باید مجموع تمامی نمایی‌ها محاسبه شود و سپس از آن برای نرمال‌سازی استفاده گردد. این وابستگی داده‌ها در نگاه اول مانعی برای موازی‌سازی به‌نظر می‌رسد. با این حال، زبان کودا و معماری GPU با قابلیت اجرای موازی هزاران نخ بصورت همزمان، این امکان را فراهم میکنند که مراحل مختلف سافت‌مکس به بخش‌های مستقل قابل موازی‌سازی شکسته شوند.

---

۱۳ . Interpretability
۱۴ . Numerical stability
۱۵ . Sigmoid
۱۶ . ReLU
۱۷ . Sequential

ارزیابی

هدف اصلی ارزیابی عملکرد تابع سافت‌مکس در لایه توجه[۱۸] شبکه مبدل بینایی[۱۹] در بستر کودا و یافتن بهینه‌ترین پیکربندی از نظر سایز بلاک و تعداد نخ‌ها است. با توجه به اینکه این تابع به‌صورت گسترده در ساختار لایه توجه شبکه‌های مبدل بینایی استفاده می‌شود و در آن بردارهای ورودی معمولاً دارای طولی برابر با ۱۹۷ هستند، اجرای سریع و بهینه آن بر روی GPU از اهمیت بالایی برخوردار است.

برای دستیابی به این هدف، سایزهای مختلفی از بلاک‌ها (نظیر ۳۲، ۶۴، ۱۲۸، ۲۵۶ و ...) مورد آزمایش قرار دهید. در هر حالت، زمان اجرای کرنل مربوط به سافت‌مکس را اندازه‌گیری کرده و نتایج را به‌صورت نموداری نمایش دهید. تحلیل این نمودار به شما کمک می‌کند تا بهترین نقطه از نظر کارایی، یعنی کمترین زمان اجرا با بیشترین بهره‌وری منابع پردازشی GPU، شناسایی شود. این نقطه به‌عنوان پیکربندی بهینه جهت استفاده در پروژه‌های بزرگ‌تر یا پیاده‌سازی سخت‌افزاری قابل استفاده خواهد بود.

منابع

[۱] "CUDA Toolkit Documentation ۱۲,۹." *Nvidia.com*, ۲۰۲۵, docs.nvidia.com/cuda/. Accessed ۱٤ May ۲۰۲۵.

[۲] freeCodeCamp.org. "CUDA Programming Course – High-Performance Computing with GPUs." *YouTube*, ۲٤ Sept. ۲۰۲٤, www.youtube.com/watch?v=۸٦FAWCzIe_٤. Accessed ۱٤ May ۲۰۲۵.

[۳] Tom Nurkkala. "CUDA Programming." *YouTube*, ۲۵ Nov. ۲۰۲۰, www.youtube.com/watch?v=xwbD٦fL٥qC۸. Accessed ۱٤ May ۲۰۲۵.

مراجع

[۱] P. Belagatti, "Understanding the Softmax Activation Function: A Comprehensive Guide," *SingleStore*, Mar. ۱۱, ۲۰۲٤. https://www.singlestore.com/blog/a-guide-to-softmax-activation-function/

توضیحات : خروجی‌های این تمرین شامل موارد زیر است:

فایل ارسالی یک فایل Zip است. خروجی‌ها برای هر سوال، در یک زیر شاخه‌ی جدا باشد. همه زیرشاخه‌ها در فایل Zip با نام HW۱_StudentNumber_Fullname.zip گنجانده شود. در صورت درخواست زمان اضافی برای انجام تمرین، ابتدا به دستیار آموزشی در تلگرام پیام دهید و پس از هماهنگی، از ۴ روز کوپن خود استفاده کنید. چنانچه سوالی دارید می‌توانید از دستیار آموزشی درس در گروه تلگرام سوال نمایید.

لطفاً حتماً از قالب آماده گزارش که قبلاً تهیه شده است، استفاده کنید. هر گونه ارسال گزارش خارج از این قالب مورد بررسی قرار نخواهد گرفت.

موفق باشید.

---

[۱۸] . Attention
[۱۹] . Vision Transformer